

Distributed Systems - 29_09

We've proven the FL3 for the perfect P2P link. We now have to prove PL1 and PL2.

Let's start by PL2.

We suppose, by contradiction, that a message m is delivered two times, at time t and t' , with $t < t'$.

Check the photo and take a look again at the proof.

- PL2: (No duplication) A message is delivered at most once
- Fact 1: The pp2p-delivery of a message is "guarded" by an if $m \notin \text{delivered}$:

The diagram shows the following code snippet with handwritten annotations:

```
upon event { sl, Deliver | p, m } do
  if m ∉ delivered then
    delivered := delivered ∪ {m};
    trigger { pl, Deliver | p, m };
```

Annotations include:

- A green arrow pointing from $t - \epsilon$ to the `if` condition.
- A blue arrow pointing from t to the `trigger` statement.
- Purple annotations: $t' - \epsilon$ and t' with a checkmark.
- Orange annotations: $t < t'$ and $9 < 10$.

- Suppose m is delivered twice, at time t and t' (with $t < t'$).
- At time t the delivery handler is executed. Since the handler is atomic we have that `delivered := delivered ∪ {m}` is executed before t' . Therefore, at t' m is in `delivered`, this contradicts the fact that `trigger { pl, deliver | P, m }` is executed at (or after) time t' .

If in the if we put the message in delivered, then in the second time we'll not be able to get into the if.

Let's now see PL1.

We prove it by contradiction. Suppose p sends m to q , and q doesn't deliver it. There could be multiple reasons why this happen:

1. q receives the message from the sl , and then it gets lost.
2. q never receives the message.

In the first case, there is a handler that checks the delivery event. If it's called, the only reason the trigger for p2pl doesn't activate is that the message has been already delivered (a message would have already entered the if, and executed the following two lines).

The second possibility is also not possible, because in that case the stubborn would be violating the second property, which is not possible.

🔗 Exercise

- Show that our stubborn algorithm does not work if we change the first property in:
 - SL1: (Stubborn-delivery) If a process p sends m to q , then q delivers m an infinite number of times. {Pay attention something is missing}.
- Show that there is no algorithm that satisfies the property above using a fair-lossy link.
- *Suppose you can use only a fixed amount k of memory (see it as having the possibility to store at most k different messages), could you implement a Perfect p2p link on a fair-lossy?

The difference in the first exercise with the original one (SL1) is the fact that "correct" is not there. If the process is not correct, then it might crash, and then the message would not be sent infinitely often (FL1), thus violating the property of the fair-lossy link.

There would be only one way, to be able to send infinitely any messages in a finite amount of time, which is not possible.

Time in Distributed Systems

We'll talk about the models of time (asynchronous, synchronous, eventually synchronous), but also the problem of clock synchronization.

Let's say we're in a synchronous system, how do we synchronize the clock (which are physical objects)?

We'll also take a look at some synchronization algorithm.

We've already seen the asynchronous one.

In the synchronous there is a bound in the delay of the channel, if a send a message in time t , then m must be received at $t + \Delta$.

The eventually synchronous works with a time unknown to us when the system becomes synchronous.

For the asynchronous system there is no global clock, with local time and no synchronization between processes. There is no bound on message delays, and the local computation steps (Exec) happen at unpredictable time.

The synchronous works with bounded delays, we can synchronize clocks. We can also assume that the local execution steps happen at a certain predetermined interval, and that takes a bounded amount of time.

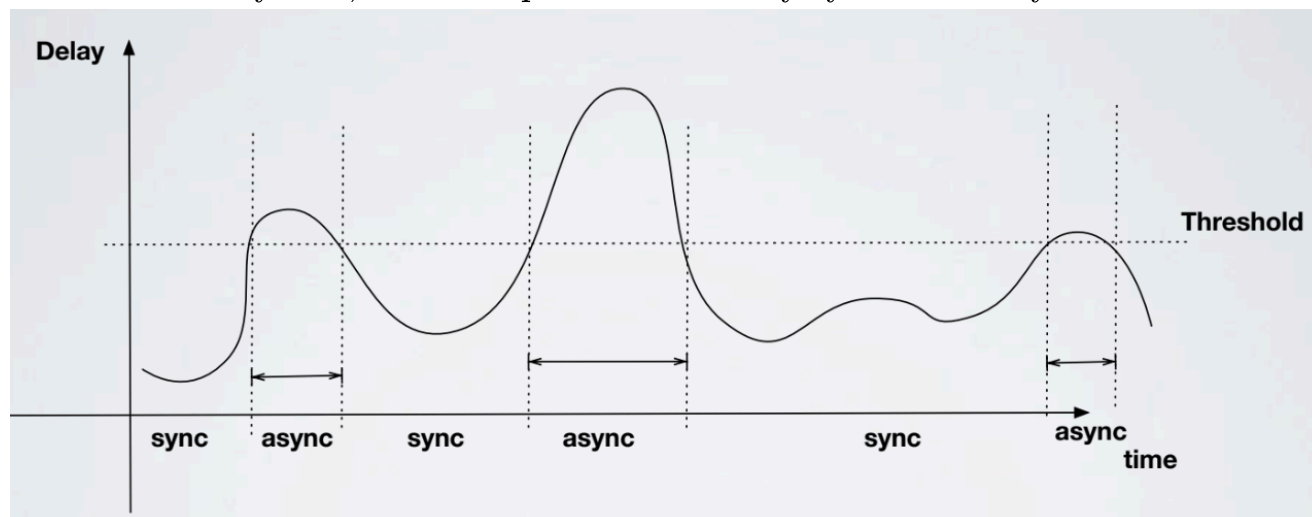
Remember that the algorithm we execute has an instant execution for definition.

A real system is synchronous most of the time.

Take the example with pinging Google. By the time it takes for the message, we may assume that the delay is 80ms (almost two times 44, the average). The problem with internet is that sometimes we may have a spike, like 740 ms.

So the internet is something that is in the middle, it's synchronous most of the time, but there might be some problems.

To model this system, we developed an eventually-synchronous system.



When we are above the threshold, we have an asynchronous system.

We know that in the future the system will stabilize, and be synchronous for long enough. Before a time T there can be oscillation, but after time T it will be synchronous forever.

A liveness property can depend on the delay of the channel in this kind of system, while a safety cannot.

So it's a system that's initially asynchronous. But there is a time t , unknown, after which the system is synchronous forever.

We have a relationship given by: $Sync \subset Eventual - Sync \subset Async$.

Regarding the possibility of being able to solve problems we have the opposite. We can solve most of the problems in a synchronous system, a bit less in e-sync and even less in async-system, it's less predictable.

Clock Synchronization

We need to synchronize clocks because many applications require an order of events, and this also includes the synchronization of time and ordering of events with time.

In a systems that is synchronous we can distinguish executions with timestamps of a message, but we need a synchronization.

There are a lot of applications that use time:

- authentication, a lot of auth algorithms uses time to avoid replay attack.
- sensors network TDMA, we divide the time in different intervals, and assign a slot to each device, without collisions.

How do we measure time? With clock. How is time measured in computers? It has a quartz crystal, and this crystal is compressed, and when it does it oscillates generating a spike of electricity. The clock showed by the O.S. is given by a formula with two constants, used to make our computer consistent with other computer's clock. The beta is the offset, used for adding/subtracting if we are in the past or future with the clock. The alpha is used if our clock is slow. So for example if our clock is half slow we use $\alpha = 2$. This kind of quick correction does not solve problem, our clock change behavior over time.

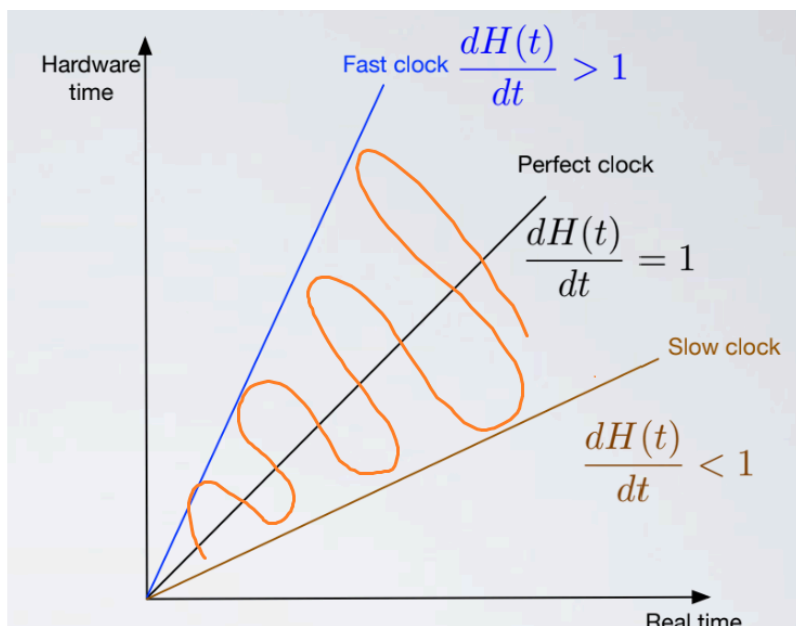
The skew is the difference between two software clocks. If a clock measures 1 PM and another 2 PM, then the skew would be 1 hour.

$$Skew_{i,j}(t) = |C_j(t) - C_i(t)|$$

We then have the drift rate, the misalignment of synchronized clocks caused by the slight inaccuracies of the time-keeping mechanisms. The real problem is this one.

We can state drift rate with $\frac{dH(t)}{dt}$. If it's equal to 1, then our clock has no drift rate. Our clock might be slow if, like, for example, 2 hours in real time passes, and 30 minutes are measured. We would have that the drift rate is $\frac{1}{4} = 0.25$,

Usually any physically clock oscillates between the two bounds, so trying to synchronize a clock only once would not work.



In the quartz in our computer, the frequency is changed even because of the temperature.

A hardware clock H is correct if its drift rate is within a limited bound of $p > 0$.
Every clock that is correct, will tell you the maximum and minimum drift rate, and the oscillation would be in that interval.

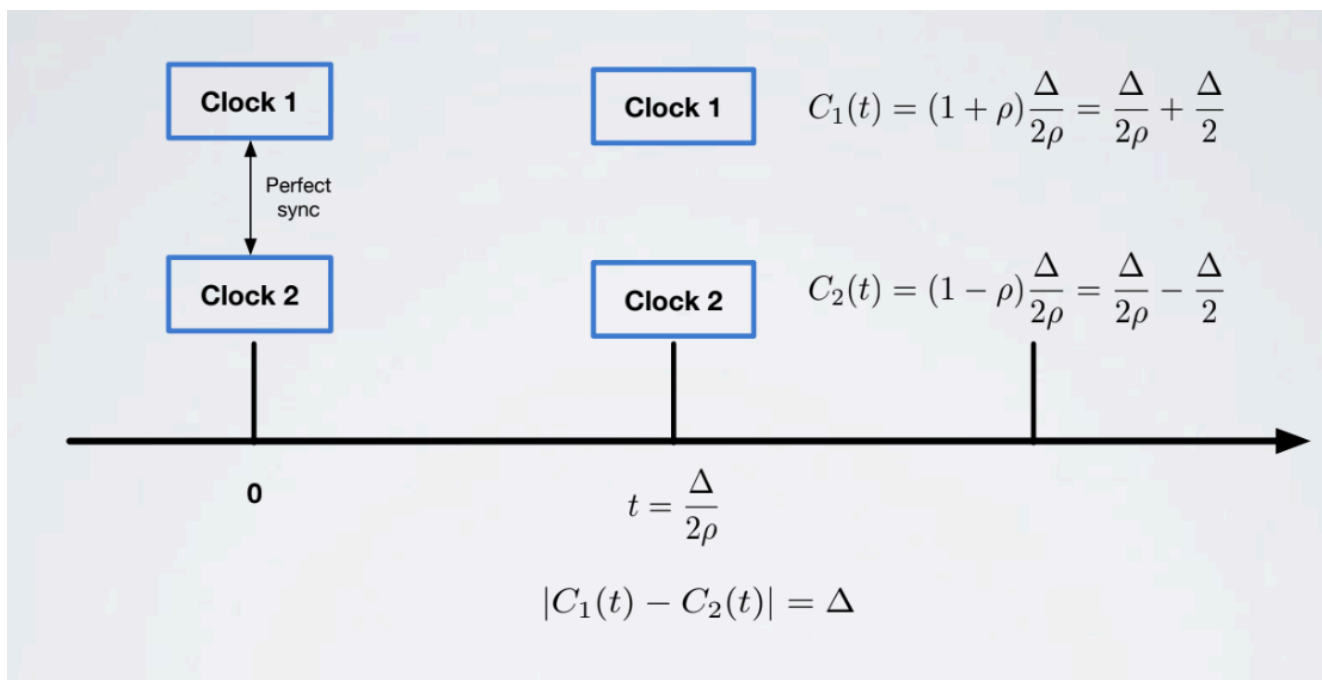
$$1 - \rho \leq \frac{dH(t)}{dt} \leq 1 + \rho$$

In presence of a correct hardware clock H we can measure a time interval $[t, t']$ (for all $t' > t$) introducing only limited errors.

$$(1 - \rho)(t_1 - t_0) \leq H(t_1) - H(t_0) \leq (1 + \rho)(t_1 - t_0)$$

Let's say $\rho = \frac{1}{2}$ and 1 hour passes. With the maximum drift we could measure 1h and 30 minutes, while with the minimum we could measure 30 minutes.

Let's now bound the skew. At first we synchronize at time zero the two clocks, C_1 and C_2 . We then take a moment $t = \frac{\Delta}{2\rho}$, the maximum drift would be possible if C_1 is super fast, while C_2 is super slow. We can then obtain the time with the formula



This equation is telling us that if we want to keep the clocks synchronize, we would have to synchronize each t , synchronizing by Δ . An example might be $\Delta = 1$.

Even if we have a perfect synchronization procedure, we have to synchronize them at intervals that depends on the skew and the drift rate of the clock.

What strategies can we use to synchronize clocks? We might move someone in the future, and in that case we would have to modify the value of β . We cannot move a clock in the past, because there would be an event with a timestamp *che ha un orario precedente al messaggio precedentemente inviato, il che non è possibile. Tipo evento alle 15:02 e poi dopo un nuovo evento con timestamp 14:58*.

Another possibility would be making the clock that is in the future slow down, till it reaches the value of another clock.

The software clock has to be monotone.

If $t' > t \implies C(t') > C(t)$, so if a real time t' is greater than t , then the software clock of t' has to be greater than t 's software clock.

To do so, we do not choose a negative β . To slow down a clock we use $\alpha < 1$, by using a masking of oscillations, as well as modifying β in order not to go back in time.

If a clock has to be monotone, we can't have $\alpha \leq 0$, otherwise we would freeze time and eventually in the future the clock of t' would be $<$ than t .

UTC, Internal and External Clock Synchronization

There is UTC (Universal Time Coordinated). It's a standard used to measure time. We have atomic clocks sending their time through radio signals or satellites.

When we synchronize with the time, we are synchronizing with UTC.

Since there exists a standard time, we can define a concept of external sync. *This set of computers, are synchronized externally with the time, like with UTC.*

The clock of each computer with authoritative external source.

With synchronized we mean that a certain clock is sync with S if

$$|S(t) - C_i(t)| < D \quad \forall i \in \{1, 2, \dots, N\}, \forall t \in I$$

So the difference is always less than D , where $D > 0$ is a synchronization bound and S the source of UTC.

A set of processes is internally synchronized if

$$|C_i(t) - C_j(t)| < D \quad \forall i, j \in \{1, 2, \dots, N\}, \forall t \in I$$

it must be true for each couple of computers in the network.

What is the relationship between these two form of synchronization?

If we are externally synchronized, we are also internally synchronized, but the opposite is not true.

If two processes are externally synchronized with a bound D , then with internal synchronization we would have a bound of $2D$.

Let's now take a look in synchronization in complete graphs.

Let's start by Cristian's algorithm, it's an external sync algorithm.

Let's say we are a process p and we have a server s . Let's say we want to synchronize now, to do so we ask to the server to give me the time. The server will send back the time. The problem with this algorithm is that it takes some time for the message containing the time to arrive. What we can do is set a timer, measuring the time it takes from the moment we sent the message to the moment it arrives, known as round trip time. If we assume that the time necessary to receive the message when sent from the server to arrive to the process is symmetric, then it would be half of the round trip time. Thus the synchronized value of the clock would be $\text{time_received} + \frac{RTT}{2}$.

The problem with this algorithm is the assumption, that delays are symmetric.

The worst case might happen when they are not actually symmetric, for example it could take a bit more to obtain the answer from the moment it was sent from the server.

Case 1

- Reply time is greater than estimate ($RTT/2$)
- Assume it is equal to $RTT - min$

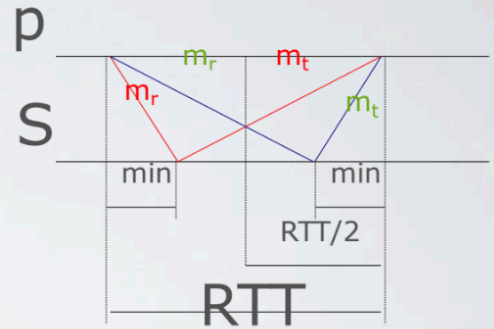
$$\Delta = RTT/2 - (RTT - min) = \frac{-RTT + 2min}{2} = -\frac{RTT}{2} + min = -\left(\frac{RTT}{2} - min\right)$$

Case 2

- Reply time is smaller than estimate ($RTT/2$)
- Assume it is equal to min

$$\Delta = \frac{RTT}{2} - min$$

Accuracy is $\pm (RTT/2 - min)$



In Case 1 we would have a time that's in the past to the actual time. The error would be $\frac{RTT}{2} - min$, where min is the minimum time it takes for the message to arrive at the server, or viceversa the time it takes for the message to arrive at the process when sent from the server (case 2).

The accuracy of the algorithm depends on the error listed above.

The problem is that the server is a single point of failure, and to solve this we use several servers.

Exercises

upon event $\langle sl, Init \rangle$ **do**

$sent := \emptyset;$

$starttimer(\Delta);$

upon event $\langle Timeout \rangle$ **do**

forall $(q, m) \in sent$ **do**

trigger $\langle fl, Send \mid q, m \rangle;$

$starttimer(\Delta);$

upon event $\langle sl, Send \mid q, m \rangle$ **do**

trigger $\langle fl, Send \mid q, m \rangle;$

$sent := sent \cup \{(q, m)\};$

upon event $\langle fl, Deliver \mid p, m \rangle$ **do**

trigger $\langle sl, Deliver \mid p, m \rangle;$

upon event $\langle pl, Init \rangle$ **do**

$delivered := \emptyset;$

upon event $\langle pl, Send \mid q, m \rangle$ **do**

trigger $\langle sl, Send \mid q, m \rangle;$

upon event $\langle sl, Deliver \mid p, m \rangle$ **do**

if $m \notin delivered$ **then**

$delivered := delivered \cup \{m\};$

trigger $\langle pl, Deliver \mid p, m \rangle;$

- PL1: (Reliable delivery) If a correct process p sends m to q , then q eventually delivers m .
- PL2: (No duplication) A message is delivered at most once.
- FL3: (No creation) If some process q delivers a message m with sender p , then m was sent by p to q .

Ragionamento personale:

Noi sappiamo che il client ha inviato la richiesta al server. Per PL1, eventualmente tale richiesta arriverà. Se è arrivata la risposta per la GET, ma il valore di v non è stato effettivamente impostato a 2, vuol dire che nel deliver nello stubborn link un messaggio è stato già flaggato come delivered, e quindi non è mai stato triggerato l'evento di deliver per il server. Il server non può essere crashato, altrimenti non avrebbe inviato la risposta. Stesso discorso vale per il secondo messaggio, il `sl` non triggera l'evento di delivery in quanto il messaggio è già stato ricevuto.

Ma la seconda ragione?

Ragionamento effettivo:

Un'esecuzione possibile è che la seconda increase è lenta, e arriva dopo la `get` (ricorda lo scheduler che fa come gli pare in contesti asincroni). A quanto pare può accadere anche in sistemi sincroni, con `inc` prima e `get` super veloci e la seconda increase lenta.

Una seconda possibilità è che, anche se l'ordine di arrivo dei messaggi è perfetto, il server risponde comunque 1. La motivazione è quella detta da me.

Per risolvere i problemi possiamo usare un numero di sequenza per ogni messaggio. Il server, a quel punto, nel caso in cui la `get` arriva prima di `inc`, aspetta la `inc` (la `get` è targata come 2, il numero di sequenza si incrementa per OGNI comando).

scrivi lo pseudocodice di un fifo link